

Informe del Taller de Introducción a Python

Fundamentos del lenguaje, estructuras de datos, control de flujo, funciones
y programación orientada a objetos

Gabriel Jaramillo Cuberos
Procesamiento de Datos (1255)

Agosto de 2026

Material analizado

01_Python_Cadenas.ipynb
02_Python_Tuplas.ipynb
03_Python_Listas.ipynb
04_Python_Conjuntos.ipynb
05_Python_Diccionarios.ipynb
06_Python_Condiciones.ipynb
07_Python_Bucles.ipynb
08_Python_Funciones.ipynb
09_Python_Clases.ipynb
Practico_Bono_1.ipynb

Este informe sintetiza los conceptos y actividades desarrollados a lo largo del material proporcionado, sin incluir la sección de pruebas, exámenes o cuestionarios de los cuadernos.

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Descripción general del taller	3
4. Desarrollo	4
4.1. Módulo 1 — Cadenas	4
4.2. Módulo 2 — Tuplas	5
4.3. Módulo 3 — Listas	5
4.4. Módulo 4 — Conjuntos	5
4.5. Módulo 5 — Diccionarios	6
4.6. Módulo 6 — Condiciones	6
4.7. Módulo 7 — Bucles	6
4.8. Módulo 8 — Funciones	7
4.9. Módulo 9 — Clases y objetos	7
5. Integración de conocimientos — Práctico Bono	8
5.1. Ejercicios iniciales	8
5.2. Nivel 1	9
5.3. Nivel 2	9
6. Relación entre los módulos	9
7. Dificultades y aspectos relevantes del aprendizaje	10
8. Conclusiones	10
9. Anexo — Organización de los archivos	10

1. Introducción

El taller presenta una introducción progresiva al lenguaje de programación Python mediante una colección de nueve cuadernos de trabajo y un práctico final. La secuencia comienza con el manejo de cadenas de caracteres y estructuras de datos fundamentales, continúa con estructuras de control y funciones, y finaliza con una introducción a las clases y los objetos.

Los materiales tienen un enfoque principalmente práctico. Cada cuaderno combina explicaciones conceptuales con fragmentos de código y ejercicios orientados a familiarizar al estudiante con la sintaxis y las características básicas de Python. El práctico bono integra varios de estos conocimientos en problemas independientes de programación.

De acuerdo con el material proporcionado, los primeros módulos trabajan cadenas, tuplas, listas, conjuntos y diccionarios; posteriormente se estudian condiciones, bucles y funciones; finalmente se aborda la programación orientada a objetos mediante clases, atributos y métodos.

2. Objetivos

2.1. Objetivo general

Comprender y aplicar los fundamentos del lenguaje Python mediante el manejo de cadenas, estructuras de datos, estructuras de control, funciones y conceptos básicos de programación orientada a objetos.

2.2. Objetivos específicos

- Manipular cadenas mediante indexación, indexación negativa, slicing, stride, concatenación y operaciones propias del tipo.
- Comprender las características y operaciones principales de tuplas, listas, conjuntos y diccionarios.
- Utilizar operadores de comparación y operadores lógicos para construir expresiones condicionales.
- Implementar ciclos `for` y `while` para repetir operaciones y recorrer estructuras de datos.
- Crear funciones reutilizables con parámetros, valores predeterminados y valores de retorno.
- Comprender el alcance de las variables y diferenciar entre variables locales y globales.
- Introducir los conceptos de clase, objeto, atributo, constructor y método.
- Integrar los conocimientos anteriores mediante ejercicios prácticos de programación.

3. Descripción general del taller

La siguiente tabla resume la progresión temática de los materiales proporcionados.

Mód.	Tema	Conceptos principales	Tiempo
1	Cadenas	Indexación, indexación negativa, slicing, stride, concatenación, secuencias de escape y operaciones con cadenas.	15 min
2	Tuplas	Creación, indexación, concatenación, slicing, anidamiento y ordenamiento.	15 min

Mód.	Tema	Conceptos principales	Tiempo
3	Listas	Indexación, contenido, slicing, modificación, operaciones, copiado y clonación.	15 min
4	Conjuntos	Contenido, operaciones con conjuntos, unión, intersección, diferencia y diferencia simétrica.	20 min
5	Diccionarios	Llaves, valores, acceso a elementos y características de los diccionarios.	20 min
6	Condiciones	Operadores de comparación, if , else , elif y operadores lógicos.	20 min
7	Bucles	range , ciclo for y ciclo while .	20 min
8	Funciones	Funciones predefinidas y definidas por el usuario, parámetros, return , valores predeterminados y alcance de variables.	40 min
9	Clases	Clases, objetos, atributos, métodos, constructor __init__ y creación de objetos.	40 min

4. Desarrollo

4.1. Módulo 1 — Cadenas

El primer módulo introduce las cadenas de caracteres como una estructura fundamental para representar texto. El material explica que los elementos de una cadena pueden consultarse mediante índices, comenzando desde el índice cero.

También se presenta la indexación negativa, que permite acceder a los elementos comenzando desde el final de la cadena. Esta característica resulta especialmente útil cuando se desea obtener los últimos caracteres sin conocer de antemano la longitud exacta del texto.

El *slicing* permite obtener una parte de una cadena especificando un intervalo de índices. A este mecanismo se añade el *stride*, que permite establecer el paso utilizado para recorrer dicho intervalo.

Entre los conceptos desarrollados se encuentran:

- Indexación desde cero.
- Indexación negativa.
- Slicing mediante rangos.
- Stride o paso de recorrido.
- Concatenación de cadenas.
- Secuencias de escape.
- Métodos y operaciones básicas sobre cadenas.

Un ejemplo representativo del acceso mediante índices es:

```
texto = "Python"

print(texto[0])    # P
print(texto[-1])   # n
print(texto[1:4])  # yth
```

El módulo establece así las bases para trabajar con información textual y para comprender posteriormente operaciones similares en otras estructuras de datos.

4.2. Módulo 2 — Tuplas

Las tuplas se presentan como estructuras que permiten almacenar una secuencia de elementos. El módulo aborda su indexación, concatenación, slicing y ordenamiento.

Un aspecto importante del contenido es el manejo de tuplas anidadas. Una tupla puede contener otras tuplas y, mediante múltiples índices, es posible acceder progresivamente a los elementos internos.

Un ejemplo sencillo de indexación es:

```
datos = ("Python", 3, ("a", "b"))

print(datos[0])
print(datos[2][1])
```

El material también muestra que las operaciones de slicing permiten construir nuevas tuplas a partir de segmentos de una tupla existente.

4.3. Módulo 3 — Listas

Las listas se estudian como colecciones ordenadas capaces de almacenar diferentes tipos de objetos. El material destaca que una lista puede contener números, cadenas, otras listas, tuplas y estructuras anidadas.

A diferencia de una estructura inmutable como una tupla, las listas pueden modificarse después de su creación. El módulo presenta operaciones de indexación, slicing, modificación y diferentes formas de copiar o clonar listas.

Por ejemplo:

```
lista = [10, "Python", 3.14]

print(lista[0])
print(lista[-1])

lista[0] = 20
print(lista)
```

El copiado y la clonación constituyen un punto importante porque permiten trabajar con una nueva lista sin confundirla con la referencia original.

4.4. Módulo 4 — Conjuntos

El cuarto módulo introduce los conjuntos como estructuras destinadas a representar colecciones de elementos y trabajar con operaciones matemáticas sobre ellas.

Se estudian operaciones como:

- Unión.
- Intersección.
- Diferencia.
- Diferencia simétrica.

Por ejemplo:

```
A = {"Python", "Java", "C++"}
B = {"Python", "JavaScript"}

print(A & B)  # Intersección
```

```
print(A | B)  # Unin
print(A - B)  # Diferencia
```

Estas operaciones permiten representar de manera directa relaciones entre colecciones de datos.

4.5. Módulo 5 — Diccionarios

Los diccionarios introducen una forma de organizar información mediante pares de llave y valor. El material señala que, a diferencia de las listas, los elementos de un diccionario se consultan mediante llaves en lugar de índices numéricos.

Una representación básica es:

```
persona = {
    "nombre": "Gabriel",
    "edad": 21
}

print(persona["nombre"])
print(persona["edad"])
```

El módulo explica que cada llave se asocia con un valor, mientras que los valores pueden ser de diferentes tipos. Las llaves deben pertenecer a tipos adecuados para ser utilizadas como identificadores dentro del diccionario.

Esta estructura resulta especialmente apropiada para representar información descriptiva donde cada dato posee un nombre o identificador.

4.6. Módulo 6 — Condiciones

Las declaraciones condicionales permiten que un programa tome decisiones dependiendo de si una expresión resulta verdadera o falsa.

El módulo desarrolla los operadores de comparación y posteriormente introduce las sentencias `if`, `else` y las estructuras condicionales relacionadas. También se presentan operadores lógicos para combinar diferentes condiciones.

Un ejemplo básico es:

```
edad = 20

if edad > 18:
    print("Puede ingresar")
else:
    print("No puede ingresar")
```

Los operadores lógicos permiten construir condiciones compuestas:

```
edad = 20
tiene_entrada = True

if edad >= 18 and tiene_entrada:
    print("Ingreso autorizado")
```

Este módulo constituye el fundamento para controlar el comportamiento de los programas según los datos recibidos.

4.7. Módulo 7 — Bucles

Los bucles permiten repetir operaciones. El módulo comienza con `range`, utilizado para generar secuencias de valores, y luego presenta los ciclos `for` y `while`.

El ciclo `for` resulta apropiado cuando se desea recorrer una secuencia o repetir una operación sobre un conjunto de elementos:

```
for numero in range(5):  
    print(numero)
```

El ciclo `while` permite repetir una operación mientras una condición se mantenga verdadera:

```
contador = 0  
  
while contador < 5:  
    print(contador)  
    contador += 1
```

El aprendizaje de los bucles permite automatizar operaciones repetitivas y prepara el terreno para la implementación de funciones más elaboradas.

4.8. Módulo 8 — Funciones

Las funciones constituyen uno de los principales mecanismos de reutilización de código. El material las define como bloques de código que realizan operaciones específicas y que pueden ser utilizados múltiples veces.

Se estudian las funciones predefinidas y las funciones definidas por el usuario. Para crear una función se utiliza la palabra reservada `def`:

```
def sumar(a, b):  
    return a + b  
  
resultado = sumar(4, 6)  
print(resultado)
```

El módulo también aborda:

- Parámetros y argumentos.
- Valores de retorno mediante `return`.
- Valores de argumentos predeterminados.
- Uso de condiciones y ciclos dentro de funciones.
- Variables globales.
- Variables locales.
- Alcance o *scope* de las variables.

La utilización de funciones permite dividir programas complejos en unidades más pequeñas, comprensibles y reutilizables.

4.9. Módulo 9 — Clases y objetos

El último módulo introduce los conceptos fundamentales de programación orientada a objetos. Se explica una clase como una estructura que sirve como modelo para crear objetos.

Los conceptos principales trabajados son:

- Clase.
- Objeto o instancia.

- Atributo.
- Método.
- Constructor `__init__`.

El material utiliza como ejemplos las clases `Circle` y `Rectangulo`. En el caso del círculo se utilizan atributos como `radius` y `color`, además de métodos como `add_radius()` y `drawCircle()`.

Una versión representativa del concepto es:

```
class Circle:
    def __init__(self, radius=3, color="blue"):
        self.radius = radius
        self.color = color

    def add_radius(self, r):
        self.radius += r

circulo = Circle(10, "red")
circulo.add_radius(2)

print(circulo.radius)
```

El módulo también desarrolla una clase para representar rectángulos y trabaja con objetos que poseen diferentes dimensiones y colores.

Como actividad de aplicación, el cuaderno propone crear una clase `Elipse` utilizando `matplotlib.patches.Ellipse`. La clase debe contar con operaciones para cambiar el ancho, cambiar el alto, modificar el color de relleno, modificar el color del borde y dibujar la elipse.

5. Integración de conocimientos — Práctico Bono

El archivo `Practico_Bono_1.ipynb` integra los conocimientos trabajados en los nueve módulos anteriores mediante ejercicios de calentamiento y problemas de dificultad progresiva.

El material identifica el práctico como parte de la materia *Procesamiento de Datos (1255)* y lo presenta como una actividad orientada a poner en práctica los conceptos aprendidos.

5.1. Ejercicios iniciales

El primer grupo de ejercicios trabaja directamente con funciones, condiciones, operadores lógicos y cadenas:

- **Menor de dos pares:** devuelve el menor de dos números si ambos son pares y el mayor si al menos uno es impar.
- **Galletas de animales:** determina si las dos palabras de una cadena comienzan con la misma letra.
- **Hace veinte:** determina si alguno de los valores es 20 o si la suma de ambos es 20.

Estos ejercicios permiten relacionar funciones con operadores de comparación y operadores lógicos.

5.2. Nivel 1

En el primer nivel se incorporan operaciones con cadenas y listas:

- **Mayúsculas:** modifica la capitalización de la primera y cuarta letra de una cadena.
- **Reversa:** invierte el orden de las palabras de una oración utilizando operaciones sobre cadenas y listas.

El ejercicio de reversa refuerza particularmente el uso de `split()`, `reverse()` y `join()`.

5.3. Nivel 2

El segundo nivel incorpora ciclos, recorridos de listas y manipulación repetitiva de cadenas:

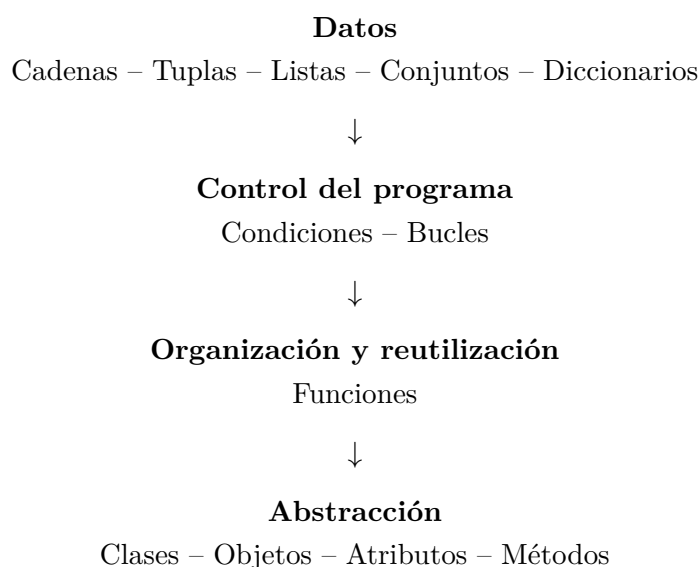
- **Problema 33:** determina si una lista contiene dos elementos 3 consecutivos.
- **Replicador:** genera una nueva cadena en la que cada carácter aparece tres veces.
- **Blackjack:** procesa tres valores entre 1 y 11, aplica las reglas indicadas y determina una suma válida o el resultado **BUST**.

Estos ejercicios representan una transición desde ejemplos aislados hacia la combinación de diferentes elementos del lenguaje.

6. Relación entre los módulos

Los contenidos presentan una progresión coherente. Las estructuras de datos proporcionan mecanismos para almacenar información; las condiciones permiten tomar decisiones sobre ella; los bucles permiten procesarla de manera repetitiva; las funciones organizan la lógica en unidades reutilizables; y las clases permiten agrupar datos y comportamiento en objetos.

La relación puede resumirse de la siguiente manera:



Esta progresión permite pasar gradualmente de operaciones básicas sobre datos a estructuras de programación con mayor nivel de abstracción.

7. Dificultades y aspectos relevantes del aprendizaje

A partir del material proporcionado pueden identificarse varios puntos que requieren especial atención durante el aprendizaje:

- **Indexación y slicing:** es necesario comprender que los índices comienzan en cero y que los rangos utilizados en slicing siguen una regla diferente a la interpretación intuitiva de “primer y último elemento”.
- **Diferencias entre estructuras:** listas, tuplas, conjuntos y diccionarios permiten almacenar datos, pero tienen características y operaciones diferentes. Reconocer cuándo utilizar cada estructura es fundamental.
- **Condiciones y operadores lógicos:** combinar expresiones mediante `and`, `or` y `not` exige comprender correctamente el valor lógico de cada condición.
- **Bucles:** es importante controlar la condición de terminación de un `while` y comprender el recorrido realizado por un `for`.
- **Alcance de variables:** las funciones introducen la diferencia entre variables locales y globales, concepto importante para evitar comportamientos inesperados.
- **Programación orientada a objetos:** distinguir entre la clase y sus instancias, así como comprender el papel de `self`, los atributos y los métodos, representa un cambio conceptual respecto a los módulos anteriores.

8. Conclusiones

El taller proporciona una ruta progresiva para adquirir fundamentos de programación con Python. Los primeros módulos permiten dominar el manejo de datos mediante cadenas y estructuras de datos; posteriormente se introducen mecanismos para controlar el flujo de ejecución y organizar el código.

El estudio de condiciones y bucles permite pasar de programas que ejecutan secuencias fijas de instrucciones a programas capaces de tomar decisiones y repetir operaciones. Sobre esta base, las funciones permiten encapsular comportamientos y reutilizar código.

Finalmente, el módulo de clases introduce una abstracción adicional al agrupar atributos y métodos dentro de objetos. La actividad de la clase `Elipse` constituye una aplicación directa de estos conceptos al combinar programación orientada a objetos con representación gráfica.

El práctico bono funciona como una integración de los conocimientos anteriores. Sus ejercicios exigen combinar cadenas, listas, condiciones, bucles y funciones, mostrando la importancia de no estudiar cada concepto de manera aislada sino como partes de un mismo lenguaje de programación.

En conjunto, los materiales proporcionan los fundamentos necesarios para continuar hacia temas de programación más avanzados, manteniendo una progresión desde operaciones simples sobre datos hasta mecanismos de abstracción y reutilización de código.

9. Anexo — Organización de los archivos

Archivo	Contenido
01_Python_Cadenas.ipynb	Operaciones y manipulación de cadenas.
02_Python_Tuplas.ipynb	Tuplas, indexación, slicing y ordenamiento.

Archivo	Contenido
03_Python_Listas.ipynb	Listas, operaciones, modificación y clonación.
04_Python_Conjuntos.ipynb	Conjuntos y operaciones lógicas entre conjuntos.
05_Python_Diccionarios.ipynb	Diccionarios, llaves y valores.
06_Python_Condiciones.ipynb	Condiciones, comparaciones y operadores lógicos.
07_Python_Bucles.ipynb	<code>range</code> , <code>for</code> y <code>while</code> .
08_Python_Funciones.ipynb	Funciones, argumentos, retorno y alcance.
09_Python_Clasas.ipynb	Clases, objetos, atributos y métodos.
Practico_Bono_1.ipynb	Ejercicios integradores de los módulos anteriores.